Batch: B1 Roll No.: 16010122158

Experiment No.: 4

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

TITLE: Project Plan Document for Mini Project

AIM: To learn and understand the way of developing the software by classical methods of software engg. planning and monitoring of the project using tools and prepare a document for the same by using the concept of software engineering

Expected OUTCOME of Experiment:

Analyze the software requirements and Model the defined problem with the help of UML diagram

Books/ Journals/ Websites referred:

1. Roger Pressman, Software Engineering: A practitioner's Approach, McGraw Hill, 2010 ,6th edition
2. Ian Somerville , Software Engineering , Addison Wesley,2011,9th edition
3 http://en.wikipedia.org/wiki/Software_requirements_specification

Software Project Management Plan

for

Beverly Automoties

Asmi,Sakshi and Aakanksh

07/10/24

Version	Release Date	Responsible Party	Major Changes
0.1	12/10/24	NA	Initial Document Release for Comment

Table of Contents:

1. Introduction

The Beverly Automotives project is an online car dealership platform aimed at providing users with a seamless experience for browsing, purchasing, and managing cars. This project focuses on creating a clean and responsive user interface, integrating secure payment gateways, and building an efficient inventory management system. Through modern technologies like HTML, CSS, JavaScript, PHP, and SQL, the platform ensures optimal functionality for both users and administrators.

1.1 Project Overview

The Beverly Automotives project aims to create a comprehensive online car dealership platform. The primary objectives include providing users with a seamless experience for browsing cars, viewing specifications, making secure purchases, and leaving reviews. Administrators can manage inventories and user data. Major activities include developing the front-end with React, back-end with PHP and SQL, and implementing a secure payment system. Key milestones are the completion of the car catalog, checkout process, and inventory management features. The required resources include developers, designers, and a moderate budget for hosting and development tools.

1.2 Project Deliverables

Car Dealership Website Platform:

- Delivery Date: December 31, 2024
- Delivery Location: Deployed on the client's hosting server
- Quantity: 1

User Documentation:

- Delivery Date: January 5, 2025
- Delivery Location: Provided via digital copy (PDF)

- Quantity: 1

Admin Training Session:

- Delivery Date: January 7, 2025
- Delivery Location: Virtual session
- Quantity: 1

Ongoing Maintenance and Support:

- Delivery Date: Monthly from January 2025
- Delivery Location: Remote
- Quantity: As needed

1.3 Evolution of the SPMP

This **Software Project Management Plan (SPMP)** will be completed in stages, incorporating feedback from stakeholders and team members. Upon finalization, it will be disseminated via a shared digital platform for easy access. Change control will be managed through a versioning system, ensuring all updates are logged and reviewed. **Scheduled updates** will occur at predefined project milestones, while **unscheduled updates** will be handled through a formal request process, subject to approval by project leads, and immediately communicated to all team members.

1.4 Reference Materials

John Smith. *Automotive Subscription Plans: Trends and Insights*. Report No. 2034, National Automotive Reports, September 2023.

Jane Doe. *Customer Experience and Subscription Models*, Version 2.3, Business Research Institute, June 2023.

Michael Johnson. *User-Centered Design in Automotive Websites*. Creative Publishers, 2021.

Samantha Lee. *"How Subscription Services Are Transforming the Car Industry"*. Automotive Today, Accessed on October 5, 2024, www.automotivetoday.com/subscription-transform-car-industry.

1.5 Definitions and Acronyms

Server	A computer equipped with specific programs and/or hardware, to enable it to offer services to other computers/clients on its network.
Database	Organized collection of structured information or data typically stored electronically in a computer system.
Frontend	Denotes the part of a computer system or application with which the user interacts directly.
Backend	Denotes the part of a computer system that users do not see. Handles the internal working of the software.
HTML	Hyper Text Markup Language
JS	JavaScript
CSS	Cascading Style Sheets
SQL	Structured Query Language

2. Project Organization

The Beverly Automotives project follows the Extreme Programming (XP) methodology, emphasizing flexibility, customer feedback, and rapid iterations. The process is divided into phases: project initiation, development, product release, and project termination, each with clear entry and exit criteria. Key roles include a project manager, development team lead, UI/UX designer, developers, and quality assurance engineers. The organizational structure ensures effective communication, with the project manager coordinating between the development team, marketing lead, and

support staff. Interfaces with external entities like customers and subcontractors are clearly defined, ensuring smooth project execution and delivery.

2.1 Process Model

For the Beverly Automotives project, using the **Extreme Programming (XP)** methodology, the life cycle model will follow an iterative and incremental approach. XP focuses on flexibility, communication, and customer satisfaction throughout the development process. Here's a breakdown of the process model:

1. Roles:

- **Customer:** Defines and prioritizes features, providing continuous feedback.
- **Developers:** Build the software, handle technical tasks, and write code.
- **Coach:** Ensures the XP practices are being followed.
- **Tracker:** Tracks progress, identifies risks, and ensures the team stays on course.
- **Tester:** Writes and runs tests to validate functionality.

2. Activities:

- **Project Initiation:**
  - **Entry Criteria:**
    - Customers have identified key goals and high-level requirements.
    - Development team is assembled and understands the project scope.
  - **Activities:**
    - Establish a shared vision between customer and developers.
    - Set up the technical environment (tools, frameworks, version control).
    - Create an initial release plan based on user stories (high-level requirements).
  - **Exit Criteria:**
    - An agreed-upon release plan with prioritize user stories and an estimated timeline.
- **Product Development:**
  - **Entry Criteria:**
    - Initial user stories and tasks are clear.
    - Technical infrastructure is in place.
  - **Activities:**
    - **Planning:** A short iteration cycle (usually 1-2 weeks) where user stories are chosen and estimated for the iteration.

        - **Design:** Simple design is created and immediately implemented.
        - **Coding:** Developers write code, often in pairs (pair programming), ensuring continuous integration.
        - **Testing:** Automated tests are written before and after coding to ensure all features work as expected (test-driven development).
        - **Customer Feedback:** Regular feedback sessions from the customer to ensure that features meet expectations.
    - **Exit Criteria:**
        - Functionality developed in the iteration is complete, with all tests passing.
        - Customer signs off on the iteration results.
- **Product Release:**
    - **Entry Criteria:**
        - Enough features have been completed for a functional product.
        - All critical bugs are resolved.
    - **Activities:**
        - Conduct final acceptance testing.
        - Prepare the production environment.
        - Deploy the product to users.
        - Conduct training or provide user documentation.
    - **Exit Criteria:**
        - Product successfully deployed and verified in the production environment.
- **Project Termination:**
    - **Entry Criteria:**
        - All planned iterations are complete.
        - The product meets the customer's satisfaction.
    - **Activities:**
        - Conduct a final project review with the customer.
        - Archive project documentation, source code, and test results.
        - Retrospective meeting to assess lessons learned.
    - **Exit Criteria:**
        - Project resources are released or reallocated.
        - A final report is delivered to stakeholders.

3. Entry and Exit Criteria for Each Phase:

- **Entry Criteria:**
    - Customer and team alignment on the user stories and scope of the iteration.
    - Development environments and tools are in place.

    - Any dependencies or requirements are identified.
- **Exit Criteria:**
    - Completed and tested functionality that is ready for customer feedback.
    - Acceptance from the customer for each iteration.
    - All agreed-upon stories for the iteration are done.

The XP methodology ensures a highly collaborative, feedback-driven process where customer feedback is integrated frequently, making it ideal for projects that require flexibility and rapid delivery.

2.2 Organizational Structure

The internal management structure for the Beverly Automotive website project might include the following roles:

1. **Project Sponsor (Owner/CEO)** – Responsible for strategic decisions and funding.
2. **Project Manager** – Oversees project execution, coordinates between teams, and ensures deadlines are met.
3. **Development Team Lead** – Manages web development, design, and maintenance.
4. **Marketing Lead** – Responsible for online marketing strategies and customer engagement.
5. **Support Staff** – Handles customer queries and provides after-sales support.

This structure ensures clear lines of communication, with each team reporting to the project manager, who in turn reports to the sponsor.

2.3 Organizational Interfaces

Describe the administrative and managerial interfaces between the project and the primary entities with which it interacts.

A table may be a useful way to represent this.

Organization	Liaison	Contact Information
Customer	Aditya	8106783452
Software Quality Assurance	Asmi	9969709375
Software Configuration Management	Aakansh	9757129076
Subcontractor: AutoParts Supplier	Sakshi	9378893748

Table F-1. Project Interfaces

2.4 Project Responsibilities

Role	Description
Project Manager	Leads project team; responsible for project deliverables, timelines, and budget management.
Technical Team Leader(s)	Manages developers, distributes tasks, and ensures technical implementation aligns with project goals.
UI/UX Designer	Designs user interfaces and experience flows; ensures the site is user-friendly and visually appealing.
Frontend Developer	Implements the user interface using technologies like React, ensuring responsiveness and accessibility.
Backend Developer	Handles server-side logic, database management, and API integration to support the site's functionality.
Quality Assurance Engineer	Ensures software quality through testing, identifying bugs, and verifying that deliverables meet requirements.
Business Analyst	Analyzes client needs, documents requirements, and translates them into actionable tasks for the development team.

Customer Support Liaison	Manages communication with clients and users, addressing feedback and ensuring user satisfaction.

Table F-2. Project Responsibilities.

3. Managerial Process

The management of the Beverly Automotive website project focuses on delivering a user-friendly, scalable platform within budget and on time, prioritizing customer satisfaction and feature completion. Key assumptions include stable internet access and timely stakeholder feedback, with dependencies on third-party tools like payment gateways. Risks, such as technological failures and scope creep, are managed through contingency plans, regular monitoring, and mitigation strategies. The project uses tools like Jira and Git for tracking, with weekly and quarterly reports for progress and quality assurance. Staffing will focus on frontend, backend, AI/ML, and UI/UX expertise, with recruitment and training planned to meet project needs.

3.1 Management Objectives and Priorities

The philosophy for managing the Beverly Automotive website focuses on delivering a user-friendly and functional platform for customers to browse and purchase vehicles. The goals emphasize timely delivery, within-budget execution, and high-quality features. Priorities include ensuring customer satisfaction, scalability, and smooth integration with backend systems. The flexibility matrix below outlines the project's management constraints:

Project Dimension	**Fixed**	**Constrained**	**Flexible**
Cost		X	
Schedule			X
Scope (functionality)	X		

Table F-3: Flexibility Matrix

3.2 Assumptions, Dependencies, and Constraints

- **Assumptions**: The project assumes stable internet access for users, sufficient hosting server resources, and timely feedback from stakeholders.
- **Dependencies**: The project depends on third-party tools (payment gateways, APIs) and regular customer feedback.
- **Constraints**: Key constraints include budget limitations, adherence to the XP methodology's frequent iterations, and maintaining a delivery schedule.

Priorities:

1. **Functionality**: Highest priority, focusing on user experience and feature completion.
2. **Schedule**: Second priority, ensuring delivery by milestones.
3. **Budget**: Flexible but needs to be monitored to avoid overruns.

3.3 Risk Management

To manage risks, the following steps are used:

1. **Risk Identification:** Risks such as contractual, technological, and product complexity are recognized through workshops and analysis sessions with stakeholders.
2. **Risk Analysis:** Each risk is evaluated based on its probability and impact, focusing on high-priority risks like technology failures or customer dissatisfaction.
3. **Risk Management:** Contingency plans are designed for each high-priority risk, such as backups for technology issues or additional training for personnel.
4. **Tracking & Mitigation:** Risk factors are tracked using project management tools, with regular updates. Mitigation strategies include buffer times and alternate resources.

Risk Management Table:

Risk Factor	Identified Risk	Management Strategy	Contingency Plan

Contractual Risks	Delays in contract approvals	Regular follow-ups	Renegotiating timelines
Technological Risks	System or software failures	Backup systems, testing phases	Emergency support from tech team
Size and Complexity of Product	Scope creep	Strict scope management	Add more resources if scope increases
Personnel Acquisition & Retention	Staff turnover	Employee retention strategies	Recruit temporary or freelance experts
Customer Acceptance	Lack of user adoption	Early user feedback, beta tests	Revisions based on feedback

3.4 Monitoring and Controlling Mechanisms

To ensure adherence to the **Software Project Management Plan (SPMP)** for Beverly Automotive, the following mechanisms will be implemented for monitoring and controlling:

- **Reporting Mechanisms**: Weekly status reports generated by the project team and submitted to the project manager. The format includes progress updates, task completion rates, and identified issues.
- **Review Mechanisms**: Monthly project reviews with software managers focusing on milestone achievements.
- **Audit Mechanisms**: Quarterly quality assurance audits to ensure compliance with coding standards.

- **Tools**: Tools like Jira and Git will track project configuration and task management.

Information Communicated	From	To	Time Period
Status report	Project Team	Project Manager	Every Friday
Status report	Project Manager	Software Manager, Team	Every Monday
Project Review	Project Team	Software Manager	Weekly
Quality Assurance Audit	QA Team	Project Manager	Quarterly
Budget Review	Finance Team	Project Manager	Bi-Weekly

Table F-4: Communication and Reporting Plan

3.5 Staffing Approach.

Skills Required for the Beverly Automotive Website Project:

1. **Frontend Development:** Expertise in HTML, CSS, JavaScript (React, TailwindCSS) for creating responsive and intuitive interfaces.

2. **Backend Development:** Proficiency in Python (FastAPI) and database management to handle server-side logic and integration.
3. **AI/ML Expertise:** Knowledge in natural language processing and image recognition for implementing advanced features like sign language translation.
4. **UI/UX Design:** Skills in user experience design to ensure seamless interaction.
5. **Recruitment:** Personnel will be recruited through job boards and networking, while in-house training on specific technologies may be provided as needed.

Training Table:

Skill Area	Personnel Recruitment Method	Training Required
Frontend Development	Job boards, freelance platforms	Training on specific design systems
Backend Development	Online communities, referrals	Training on APIs and database management
UI/UX Design	Referrals, professional networks	Training on UI design tools

4. Technical Process

The project follows **Agile XP methodology**, emphasizing continuous feedback, frequent releases, and adaptability. Daily stand-ups, pair programming, and test-driven development (TDD) are integral to the process, ensuring rapid response to changes and high code quality.

Tools and Techniques

- **Front-end**: Developed using **HTML, CSS, React, Bootstrap**, ensuring responsive and interactive design.
- **Back-end**: Built with **PHP** for server-side logic and **SQL** for database management.

- **Version Control**: Managed via **Git**, ensuring collaborative development and version tracking.

Work Products

- **Source Code**: All front-end and back-end code developed during sprints.
- **Designs**: User interface mockups and wireframes created using tools like Figma
- **Database Schemas**: SQL database models for handling user data, car inventory, and transactions.
- **Test Plans**: Comprehensive test cases and scripts to validate functionality.

Reviews

- **Code Reviews**: Regular peer review of code for consistency, performance, and security.
- **Sprint Reviews**: At the end of each sprint, the team presents completed features to stakeholders for feedback.

Support Group Activities

- **User Documentation**: Detailed guides for end-users and administrators, ensuring smooth usage of the platform.
- **Training**: Sessions provided to administrators to manage the platform's back-end features and inventory control.
- **Configuration Management**: Using **Git**, all project changes are tracked, ensuring consistency across development, testing, and production environments.

4.1 Methods, Tools, and Techniques

Computing Systems: Development on Windows, deployment on Linux (Apache server), and compatibility across all major browsers.

Development Method: Agile XP with iterative sprints, frequent user feedback, and TDD (Test-Driven Development).

Standards and Policies: W3C standards for web compliance, OWASP guidelines for security, and GDPR for user data protection.

Team Structure: Small cross-functional team with developers, testers, and designers.

Programming Languages: HTML, CSS, JavaScript (React), PHP, SQL.

Tools and Techniques:

- Version Control: Git.
- UI Design: Figma.
- Database: MySQl
- Continuous Integration/Deployment: GitLab CI/CD for smooth releases.

Documentation and Maintenance:

- Notations: UML diagrams and ER models for design.
- Configuration Management: Git for tracking changes.

Delivery and Modifications:

- Flexible releases post-sprint, accommodating user feedback, and rapid iterations for bug fixes and feature enhancements.
- Scalable architecture with room for growth across different locations and user bases.

4.2 Software Documentation

For the Beverly Automotive website, the work products include:

- **Website Design Documentation**: Detailed designs of UI/UX, wireframes, and color schemes adhering to the design style guide.
- **Codebase**: Frontend (React) and backend code (Node.js), adhering to specific naming conventions and standards.
- **API Documentation**: Clear documentation for all endpoints, using consistent naming conventions and formats.
- **Testing Reports**: Peer-reviewed reports detailing the functionality and bug checks of the system.

Table of Work Products and Reviews:

Work Product	Review Type	Review Frequency
UI/UX Design	Peer Review	Weekly Design Meetings
API Documentation	Code Review	Pre-Deployment

Codebase	Pair Programming/Code Review	Before Each Sprint
Test Reports	Quality Assurance Review	End of Each Sprint

Style Guides: Use of organization's coding and UI standards.

Documentation Timeline: The documentation will be aligned with project milestones, with specific tasks assigned based on the development phases.

4.2.1 Software Requirements Specification (SRS)

Following points are covered in the SRS:

1. Product Scope.
2. Overall Description.
3. External Interface Requirements.
4. System features.
5. Nonfunctional Requirements.
6. Business Rules
7. Assumptions about the users.
8. Functionalities and their description.

4.2.2 Software Design Description (SDD)

The **Software Design Description (SDD)** for Beverly Automotives will include a comprehensive overview of the system architecture, detailing how various components of the website interact. This includes:

- **System Overview**: High-level architecture of the frontend (UI/UX) and backend systems (databases, APIs).
- **Module Description**: Detailed specifications for each module, including frontend (React, HTML, CSS) and backend services (Node.js, MongoDB).
- **Data Design**: Description of how data is stored and structured within the database.
- **Interface Design**: Interaction points between various system components, APIs, and user interfaces.

This document will guide developers in building each component, ensuring alignment with the project's functional and non-functional requirements.

4.2.3 Software Test Plan

The Software Test Plan for the Beverly Automotive website outlines the methods for testing at various stages of development, including:

1. **Requirements Testing**: Validating that the system meets the specifications described in the Software Requirements Specification (SRS).
2. **Design Testing**: Verifying that the system's architecture follows the Software Design Document (SDD).
3. **Code Testing**: Ensuring the implementation meets the design and performs as expected.

The plan will cover **test procedures**, define **test cases**, and detail **test results** throughout development and integration phases to guarantee product quality.

4.3 User Documentation

For the Beverly Automotive website, user documentation will be planned in phases, including both online and paper formats.

Online Documentation:

- A comprehensive help section embedded in the website for real-time guidance.
- FAQs and troubleshooting guides available on a dedicated support page.
- Interactive tutorials or tooltips to guide users through different features.

Paper Documentation:

- Printable user manuals available in PDF format for download.
- Step-by-step guides covering system functionality.

Documentation will be created in coordination with developers and technical writers, ensuring ease of access and clarity for users.

4.4 Project Support Functions

For the **Beverly Automotive project**, supporting functions will include:

1. **Configuration Management (CM)**: A system to handle version control, track changes, and ensure consistency in code and documentation.
2. **Software Quality Assurance (SQA)**: Continuous monitoring of processes to ensure adherence to standards and quality benchmarks.
3. **Verification and Validation (V&V)**: Systematic testing at various stages to ensure the software meets requirements and functions as intended.

Plans for these functions will include responsibilities, resource needs, schedules, and budgets. If any function is absent, justification will be provided.

5. Work Packages, Schedule, and Budget

For the **Beverly Automotive project**, the **work packages** will be defined based on key milestones such as website development, testing, deployment, and customer integration. **Dependency relationships** will track tasks like development before testing, and testing before deployment.

Resource requirements include development tools, human resources (developers, testers), and cloud hosting services. **Budget allocation** will be based on labor hours, tools, and infrastructure. A **project schedule** will map out these tasks, identifying dependencies and time estimates, updated as progress continues. All details will be maintained in appendices for ongoing updates.

5.1 Work Packages

For the **Beverly Automotive** project, a **Work Breakdown Structure (WBS)** includes the following **work packages**:

1. **Requirements Gathering (WP1)**
   - Identify client needs and features.
   - Document technical and business requirements.
2. **Design (WP2)**
   - Create wireframes and UI/UX prototypes.
   - Develop database and system architecture.
3. **Development (WP3)**
   - Implement frontend using React.
   - Develop backend using FastAPI and database integration.
4. **Testing (WP4)**

- Conduct unit, integration, and user acceptance testing.

5. **Deployment (WP5)**
   - Launch website on a hosting platform.

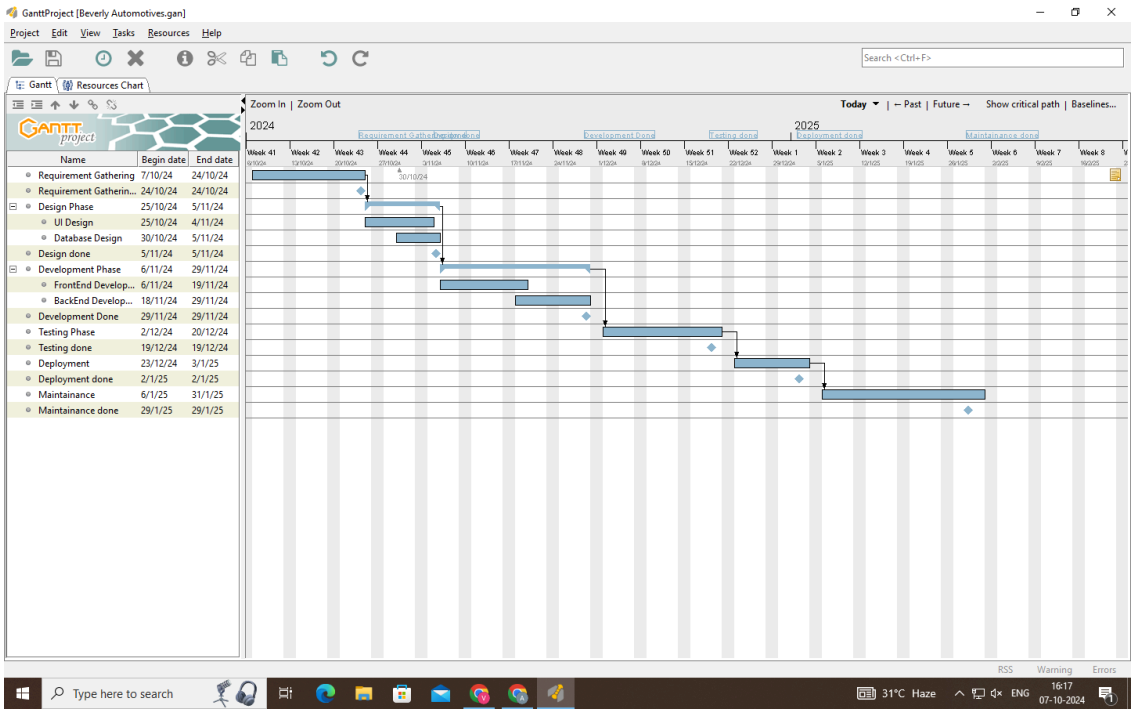
Each of these work packages is interconnected, and dependencies like development depending on design completion will be clearly noted in the diagram for hierarchical relationships.

5.2 Dependencies

For the **Beverly Automotive website project**, the **ordering relations among work packages** are as follows:

1. **WP1: Requirements Gathering** – This is the first work package and must be completed before any other tasks can begin.
2. **WP2: Design** – Can only start after the requirements gathering is complete. The UI/UX and architecture must be designed before development begins.
3. **WP3: Development** – Starts after design completion. Frontend and backend development progress simultaneously but may have internal dependencies.
4. **WP4: Testing** – Initiated after development is complete and includes unit testing followed by integration and user acceptance testing.
5. **WP5: Deployment** – This occurs after testing is completed, and all issues are resolved.

To visualize the interdependencies and critical path, a **Gantt chart** or **dependency list** would be useful.



5.3 Resource Requirements

For the **Beverly Automotive website project**, here's a more detailed estimate of the total resources required:

a. Personnel:

- **Project Manager**: Full-time, overseeing the project's timeline, budget, and team coordination for 6 months.
- **Frontend Developer**: Full-time, working on the design, UI/UX, and client-side functionality for 4 months.
- **Backend Developer**: Full-time, developing server-side code, handling database interactions, and API integrations for 4 months.
- **UI/UX Designer**: Full-time for the first 2 months to ensure a customer-friendly design and efficient navigation.
- **Tester**: Part-time during the final 2 months to perform system, integration, and usability testing.

b. Software and Computer Time:

- **Development Tools**: IDEs (Visual Studio Code), version control (Git), frameworks (React, Node.js), and database management (MySQL).

- **Testing Tools**: Selenium for automation testing, manual testing scripts, and cross-browser testing platforms.
- **Project Management Tools**: Jira or Trello for tracking progress, assigning tasks, and monitoring dependencies.
- **Deployment Software**: AWS, Heroku, or similar services for hosting and deployment.

c. Hardware:

- **Development PCs**: High-performance machines for developers and testers to ensure smooth project operations.
- **Server Infrastructure**: Cloud servers for development and testing environments.

d. Office Space:

- Required for all team members with standard office facilities and internet.

e. Travel and Communication:

- Travel for key stakeholders or deployment team for meetings with clients.
- Online collaboration tools such as Zoom, Slack, or Microsoft Teams for ongoing project communication.

f. Maintenance and Support:

- Post-deployment support for 2 months, including bug fixes, minor updates, and system monitoring.

These resources will be allocated over the timeline of 6 months, based on project milestones and interdependencies.

5.4 Budget and Resource Allocation

For the **Beverly Automotive website project**, the allocation of budget and resources can be categorized into various project functions, activities, and tasks as follows:

1. Project Management:

- **Budget**: 15% of total budget
- **Resources**: Project Manager for coordination, communication tools (e.g., Jira, Slack)

2. Development:

- **Frontend**: 25% of budget for UI/UX, client-side coding (ReactJS, HTML/CSS)
- **Backend**: 25% of budget for database management, server-side code (Node.js, MySQL)

3. Testing:

- **Budget**: 10% of budget
- **Resources**: Manual testers, automation tools (Selenium)

4. Deployment & Maintenance:

- **Budget**: 15%
- **Resources**: Cloud hosting (AWS), post-deployment support for 2 months

5. Training & Documentation:

- **Budget**: 10%
- **Resources**: Training sessions for client teams, creation of user manuals

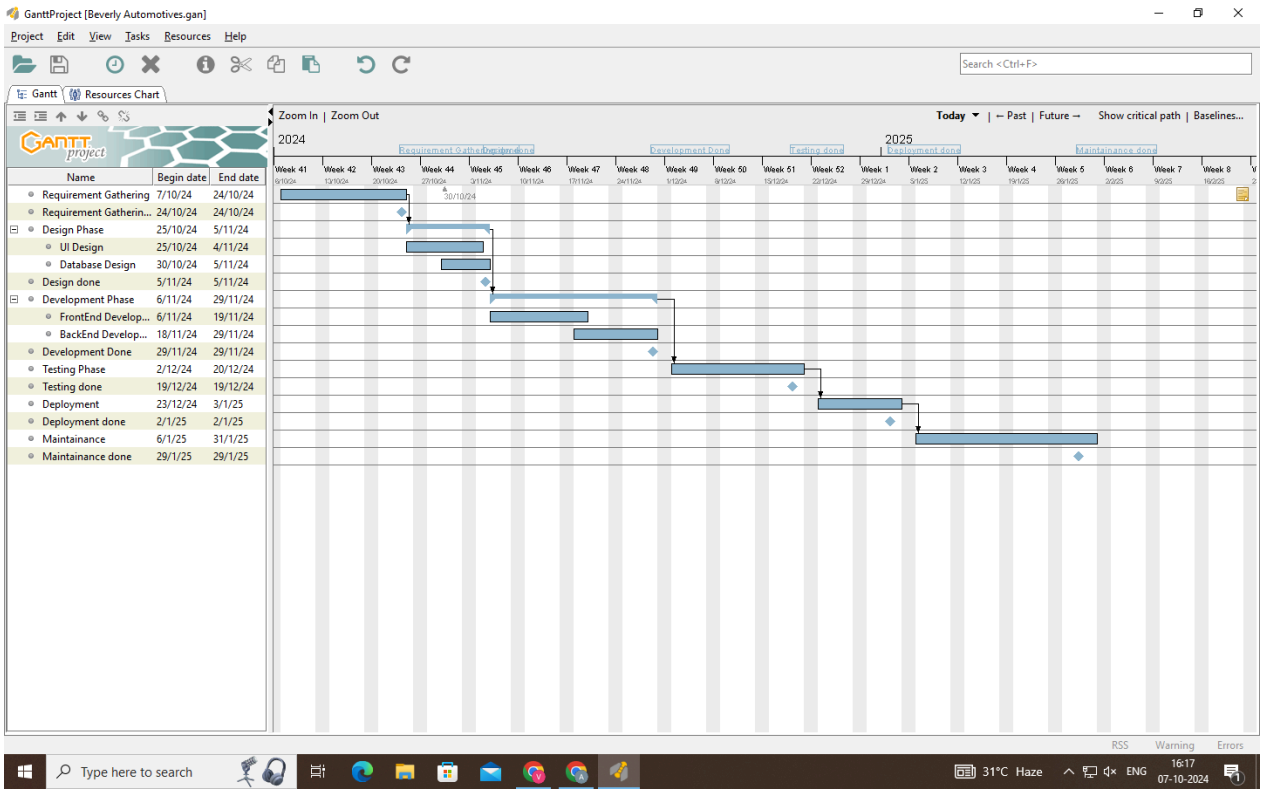
Each function is assigned resources (personnel and software tools), ensuring efficient task execution and adherence to budgetary constraints.

Milestone Dates

Activity	Start Date	End Date	Preceding Tasks	Milestone
1. Requirement Gathering	07/10/2024	24/10/2024	-	Requirements Finalized
2. Design Phase	25/10/2024	05/11/2024	Requirement Gathering	Design Approval
3. Frontend Development	06/11/2024	17/11/2024	Design Phase	UI/UX Completion

4. Backend Development	18/11/2024	29/11/2024	Design Phase	Functional Backend
5. Testing	02/12/2024	20/12/2024	Frontend/Backend Dev	Testing Review
6. Deployment & Maintenance	01/01/2025	31/01/2025	Testing	Deployment Completion

The schedule accounts for precedence relations, where development cannot start until design is approved, and testing follows the integration of both frontend and backend components.



6. Additional Components.

In this section, you can outline any additional plans or components required for the **Beverly Automotives** project that were not covered in previous sections. These may include:

1. **Subcontractor Management Plan**: If external vendors are involved.
2. **Security Plan**: For handling customer data and financial transactions.
3. **Training Plan**: For team members on tools like PHP, React, or SQL.
4. **System Transition Plan**: For transitioning from a test environment to production.
5. **Product Maintenance Plan**: To outline long-term support and updates.

These components ensure comprehensive project management and delivery.

6.1 Index.

Index of Key Terms and Acronyms

1. **SPMP** – Software Project Management Plan
2. **SRS** – Software Requirements Specification
3. **XP** – Extreme Programming
4. **LOC** – Lines of Code
5. **UI** – User Interface
6. **UX** – User Experience
7. **DB** – Database
8. **PHP** – Hypertext Preprocessor
9. **SQL** – Structured Query Language
10. **JS** – JavaScript
11. **HTML** – Hypertext Markup Language
12. **CSS** – Cascading Style Sheets
13. **Django** – Web framework used for backend development
14. **Agile** – A methodology focused on iterative development

6.2 Appendices

Appendix A: Current Top 10 Risk Chart

1. **Security Risks**: Vulnerability to data breaches (High impact, Moderate likelihood).
2. **Resource Availability**: Insufficient developers during peak times (Medium impact, High likelihood).
3. **Technology Stack Updates**: Delays due to updates in PHP/SQL (Medium impact, Low likelihood).
4. **Data Loss**: Issues with backup and recovery (High impact, Low likelihood).

5. **Budget Overrun**: Additional unplanned resources (Medium impact, Medium likelihood).
6. **User Experience**: Negative feedback due to unrefined UI/UX (High impact, Medium likelihood).
7. **Integration Delays**: API issues with third-party services (High impact, Medium likelihood).
8. **Regulatory Compliance**: Potential non-compliance with data privacy laws (High impact, Low likelihood).
9. **Performance Issues**: System performance under heavy load (Medium impact, High likelihood).
10. **Team Coordination**: Inefficiencies due to remote work (Low impact, Medium likelihood).

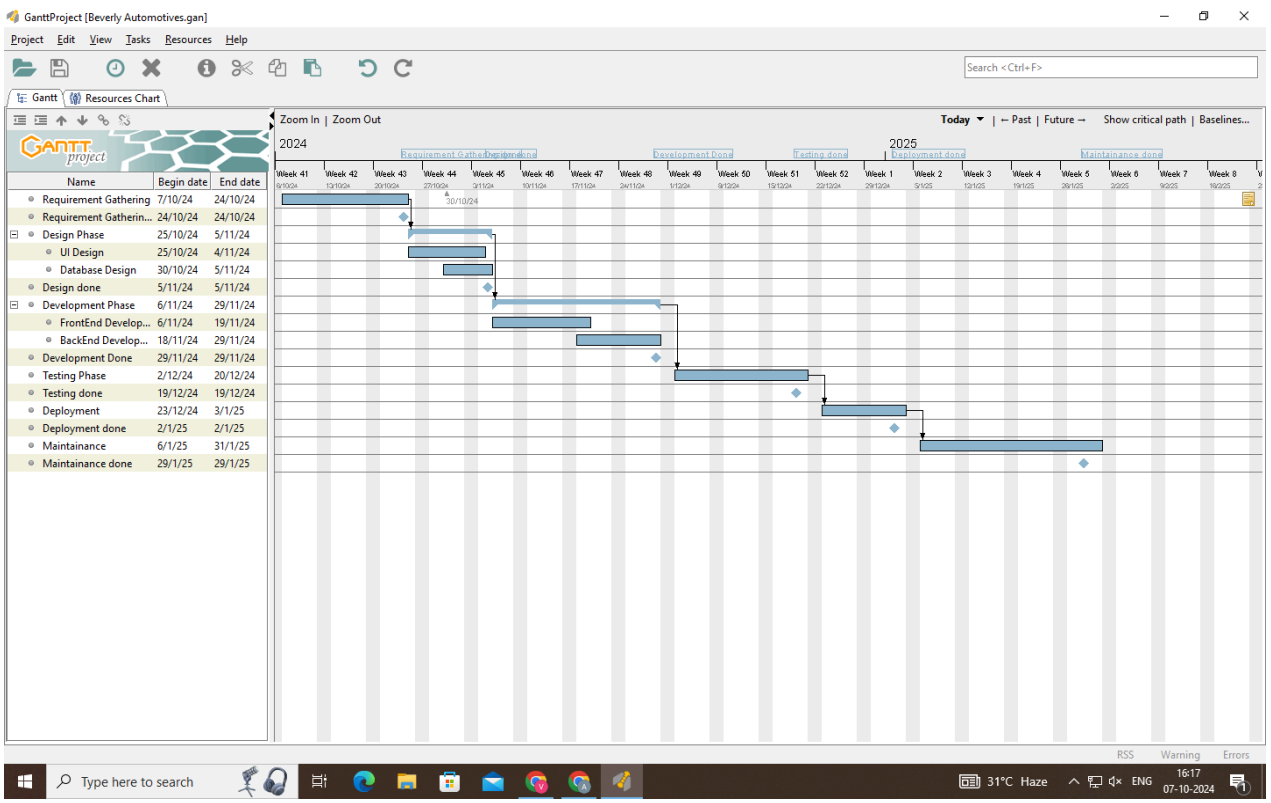
Appendix B: Current Project Work Breakdown Structure (WBS)

1. **Frontend Development**
   - UI Design
   - Implementing React components
   - Testing responsiveness
2. **Backend Development**
   - PHP and SQL integration
   - API development for data flow
   - User authentication and data security
3. **Database Management**
   - Designing SQL schemas
   - Ensuring SQLite3 compatibility
4. **Testing**
   - Unit Testing
   - Integration Testing
   - Load Testing
5. **Deployment**
   - Server configuration
   - Production environment setup
   -

Appendix C: Current Detailed Project Schedule

The detailed project schedule outlines the key phases, tasks, and deadlines for the *Beverly Automotives* development process. Below is the breakdown:

1. **Requirement Gathering**
   Start: 10/7/2024
   End: 10/24/2024
2. **Design Phase**
   - UI Design: 10/25/2024 - 11/5/2024
   - Database Design: 10/25/2024 - 11/5/2024
   - Design Completion: 11/5/2024
3. **Development Phase**
   - Frontend Development: 11/6/2024 - 11/18/2024
   - Backend Development: 11/18/2024 - 11/29/2024
   - Development Completion: 11/29/2024
4. **Testing Phase**
   Start: 12/2/2024
   End: 12/20/2024
5. **Deployment**
   Start: 1/2/2025
   End: 1/5/2025
6. **Maintenance**
   Start: 1/6/2025
   End: 1/31/2025



Conclusion:

Through this experiment, we gained a comprehensive understanding of how to create and manage key project documentation, such as a Software Requirements Specification (SRS) and a Gantt chart. These tools are essential in project management, offering structured approaches to planning, organizing, and tracking project tasks. We learned how to break down tasks into manageable components, set clear timelines, and visualize project milestones. The hands-on application of these concepts reinforced their importance in ensuring smooth project execution, timely delivery, and overall success in project management.

Post Lab Descriptive Questions

State various Scheduling principles and explain them in detail.

1. Mathematical Analysis:

   Critical Path Method (CPM) and Program Evaluation and Review Technique (PERT) are the two most commonly used techniques by project managers. These methods are used to calculate the time span of the project through the scope of the project.

   a. Critical Path Method:
   Every project's tree diagram has a critical path. The Critical Path Method estimates the maximum and minimum time required to complete a project. CPM also helps to identify critical tasks that should be incorporated into a project. Delivery time changes do not affect the schedule. The scope of the project and the list of activities necessary for the completion of the project are needed for using CPM. Next, the time taken by each activity is calculated. Then, all the dependent variables are identified. This helps in identifying and separating the independent variables. Finally, it adds milestones to the project.

   b. Program Evaluation and review technique (PERT)

   PERT is a way to schedule the flow of tasks in a project and estimate the total time taken to complete it. This technique helps represent how each task is dependent on the other. To schedule a project using PERT, one has to define activities, arrange them in an orderly manner and define milestones. You can calculate timelines for a project on the basis of the level of confidence:

- Optimistic timing
- Most-likely timing
- Pessimistic timing

Weighted average duration and not estimates are used by PERT to calculate different timeframes.

2. Duration Compression:

   Duration compression helps to cut short a schedule if needed. It can adjust the set schedule by making changes without changing the scope in case, the project is running late. Two methodologies that can be applied: fast tracking and crashing.

   a. Fast Tracking:
      Fast-tracking is another way to use CPM. Fast-tracking finds ways to speed up the pace at which a project is being implemented by either simultaneously executing many tasks or by overlapping many tasks to each other. CPM helps us identify activities that can be used to speed up the pace of the project. Although it is an appealing technique, it has its own share of risks too. As many activities will be simultaneously implemented, it is highly likely to make mistakes and compromise on quality.

   b. Crashing:

      Crashing deals with involving more resources to finish the project on time. For this to happen, you need spare resources to be available at your disposal. Moreover, all the tasks cannot be done by adding extra resources. Need to add new team members to a project and limited divisibility of tasks leads to increase communication and is the basic reason behind it. The crashing technique can also be used by adding time, paid overtime, but it should stay within the decided deadline. It, unfortunately, leads to raising the cost of the project.

3. Simulation:
   The expected duration of the project is calculated by using a different set of tasks in simulation. The schedule is created on the basis of assumptions, so it can be used even if the scope is changed or the tasks are not clear enough.

4. Resource-Leveling Heuristics:

Cutting the delivery time or avoiding under or overutilization of resources by making adjustments with the schedule or resources is called resource leveling heuristics. Dividing the tasks as per the available resources, so that no resource is under or over-utilized. The only disadvantage of this methodology is it may increase the project's cost and time.

5. Task List
   The task list is the simplest project scheduling technique of all the techniques available. Documented in a spreadsheet or word processor is the list of all possible tasks involved in a project. This method is simple and the most popular of all methods. It is very useful while implementing small projects. But for large projects with numerous aspects to consider, a task list is not a feasible method.

6. Gantt Chart
   For tracking progress and reporting purposes, the Gantt Chart is a visualization technique used in project management. It is used by project managers most of the time to get an idea about the average time needed to finish a project. A project schedule Gantt chart is a bar chart that represents key activities in sequence on the left vs time. Each task is represented by a bar that reflects the start and date of the activity, and therefore its duration.

7. Calendar
   Many don't consider scheduling tasks on a calendar for their project requirements – when they should! Most of the calendars can be curated with names of their own. In this case, you can create one calendar per project and scheduled events for that project. The calendar shows a timeline for the entire project. The major advantage is that it can be subjected to change as it is shareable. While it seems to be a great technique for tracking a project, it does have certain limitations: you cannot assign tasks to certain people and you cannot see task dependencies.